

 Serviço Nacional de Aprendizagem Industrial Santa Catarina	AVALIAÇÃO PRÁTICA		Desempenho
	Data: 26/08/2025		
	Docente: Marcio Schoenfelder		
	Curso Técnico em Desenvolvimento de Sistemas		
	Unidade Curricular: Programa o Orientada a Objetos		
	Turma: QA IDM 2025/1 V1		
	Estudante: David Luis Kim		

CAPACIDADES
Aplicar fundamentos básicos de orientação a objetos no Dart, incluindo criação de classes, atributos, métodos e encapsulamento simples.

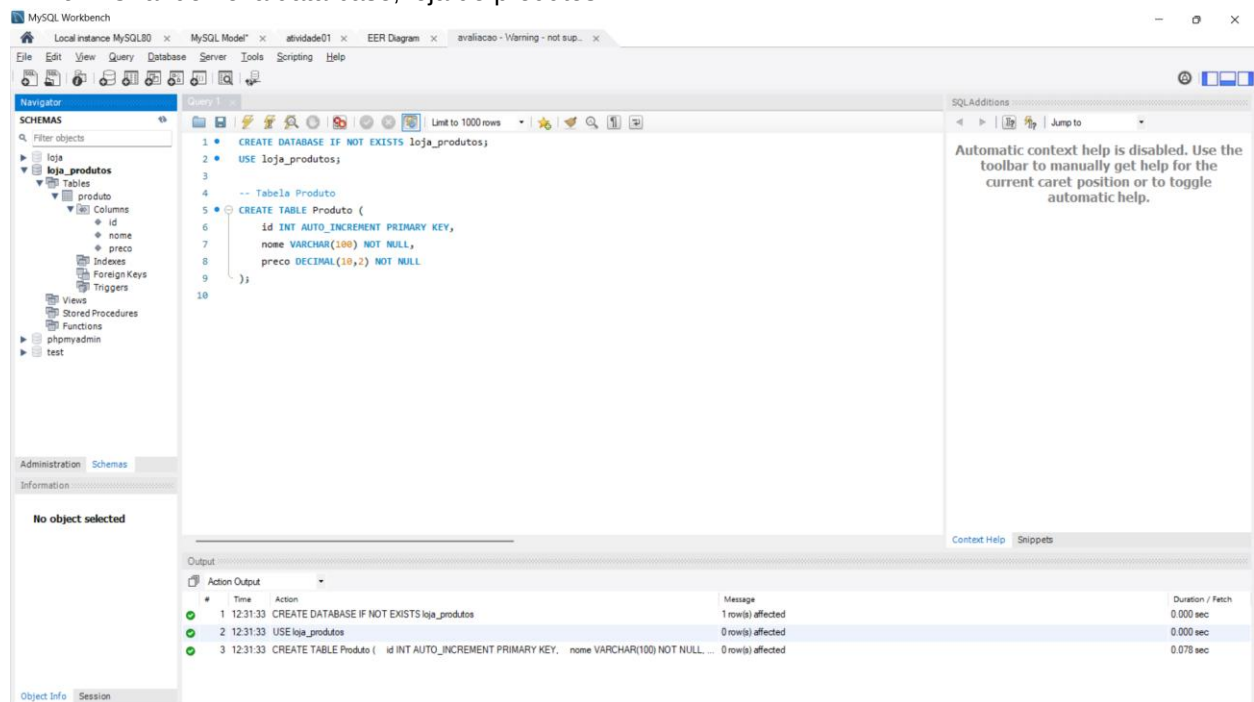
CONTEXTUALIZAÇÃO
Você, desenvolvedor de TI, foi contratado para desenvolver um sistema simples de cadastro de produtos para uma pequena loja, O sistema deve permitir criar produtos, definir seus preços e exibir suas informações.

DESAFIO
Cenário: Você deverá desenvolver um programa em Dart que atenda aos seguintes requisitos: • Criar a classe produto com atributos: nome (String) e preco (double). • Tornar o atributo preco privado, usando getter e setter par acessar/modificar o valor. • No setter, validar para que o preço não seja negativo. • Criar um método exibir() que mostre nome e preço no console. • No método main(), criar pelo menos 2 produtos, definir os preços e exibir as informações.

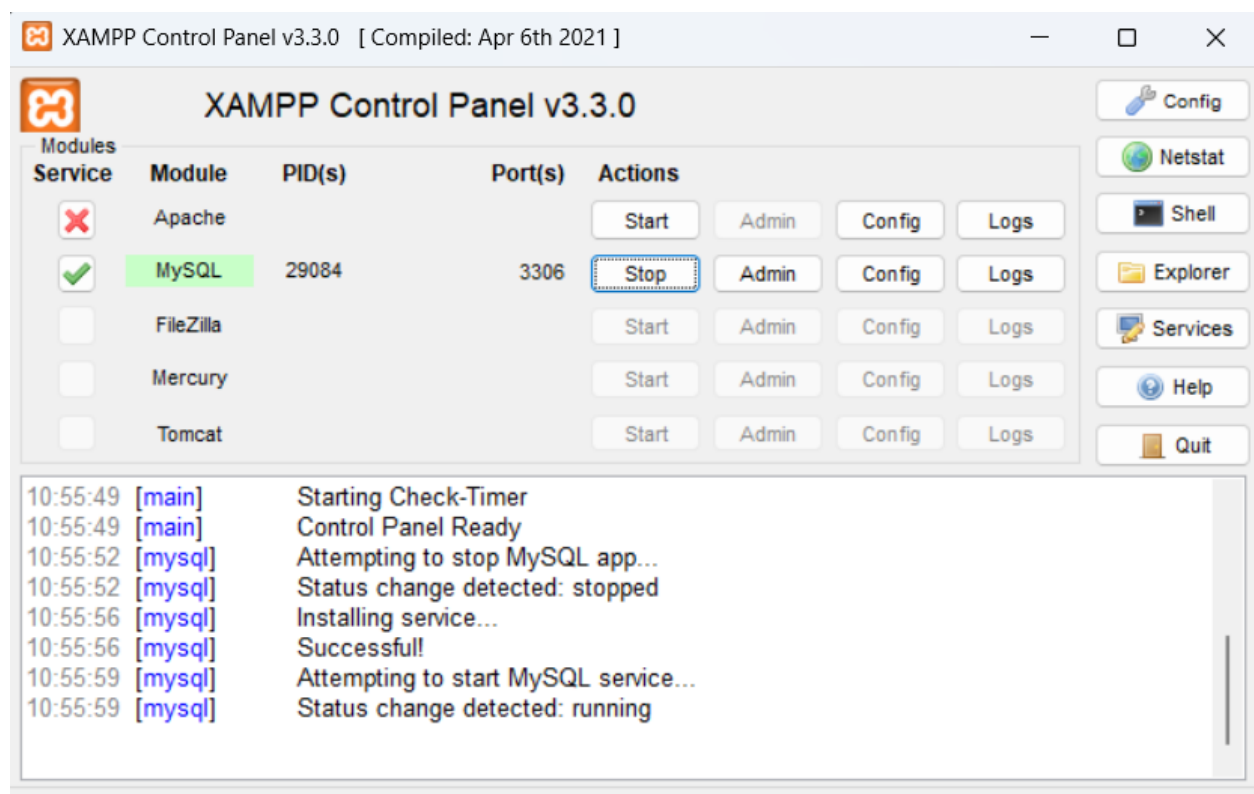
RESULTADOS E ENTREGAS
• Código dart funcional com uso de classe, atributos, métodos e encapsulamento. • Saída clara no terminal com os dados dos produtos • Boas praticas de codificação nomes claros, indentação e comentários simples
Entregáveis • Código conte .dart • Prints da tela com saída do programa.

PRINTS DE COMPROVAÇÃO

Print 1: Criando nova data base, loja de produtos.



Print 2: Iniciando Xampp para novo banco de dados.



Print 3: Conexão do .dart no vscode para o banco de dados:

```
/// -----  
/// Conexão com MySQL  
/// -----  
Future<MySQLConnection> getConnection() async {  
  final settings = ConnectionSettings(  
    host: 'localhost',  
    port: 3306,  
    user: 'davidkim',  
    password: 'senha',  
    db: 'loja_produtos',  
  );  
  return await MySQLConnection.connect(settings);  
}
```

Print 4: Classe produto com string nome, e _preco double.

```
/// -----  
/// Classe Produto  
/// -----  
class Produto {  
  String nome;  
  double _preco;  
}
```

Print 5: Construtor com validação de preço e getter e setter com validação. + método funcional para exibir produto e preço.

```
/// Construtor com validação de preço  
Produto({required this.nome, required double preco})  
  : _preco = preco >= 0 ? preco : 0 {  
    if (preco < 0) {  
      print('Preço negativo não permitido! Definido como 0.');    }  
  }  
  
/// Getter do preço  
double get preco => _preco;  
  
/// Setter do preço com validação  
set preco(double valor) {  
  if (valor >= 0) {  
    _preco = valor;  
  } else {  
    print('Preço negativo não permitido! Valor não alterado.');  }  
}  
  
/// Método para exibir informações do produto  
void exibir() {  
  print('Produto: $nome | Preço: R\${_preco.toStringAsFixed(2)}');}
```

Print 6: Funções separadas para manipular o banco.

```
/// -----  
/// Funções para manipular banco  
/// -----  
  
/// Insere um produto no banco  
Future<void> inserirProduto(Produto produto) async {  
  final conn = await getConnection();  
  await conn.query(  
    'INSERT INTO Produto (nome, preco) VALUES (?, ?)',  
    [produto.nome, produto.preco],  
  );  
  await conn.close();  
}  
  
/// Lista todos os produtos do banco  
Future<void> listarProdutos() async {  
  final conn = await getConnection();  
  var results = await conn.query('SELECT * FROM Produto');  
  
  for (var row in results) {  
    print('Produto #${row[0]} - ${row[1]} | R\${row[2]}');  
  }  
  await conn.close();  
}
```

Print 7: método main(), criar pelo menos 2 produtos, definir os preços e exibir as informações.

```
/// -----  
/// Função principal  
/// -----  
Run | Debug  
void main() async {  
    // Criando produtos  
    Produto produto1 = Produto(nome: 'Notebook', preco: 3500.00);  
    Produto produto2 = Produto(nome: 'Mouse', preco: 150.00);  
  
    // Tentativa de definir preço negativo (testa validação)  
    produto2.preco = -50.00; // Não será alterado  
    produto1.preco = 3600.00; // Alteração válida  
  
    // Exibindo no console (OOP)  
    print('--- Produtos Criados (OOP) ---');  
    produto1.exibir();  
    produto2.exibir();  
  
    // Inserindo no banco de dados  
    await inserirProduto(produto1);  
    await inserirProduto(produto2);  
  
    // Listando produtos do banco  
    print('\n--- Produtos no Banco de Dados ---');  
    await listarProdutos();  
}
```

Print 8: Primeiro dart run no console:

```
Preço negativo não permitido! Valor não alterado.  
--- Produtos Criados (OOP) ---  
Produto: Notebook | Preço: R$3600.00  
Produto: Mouse | Preço: R$150.00  
  
--- Produtos no Banco de Dados ---  
Produto #1 - Notebook | R$3600.0  
Produto #2 - Mouse | R$150.0  
  
Exited.
```

Print 9: verificação dos dados no banco

12 • `Select * from produto;`

Result Grid			
Filter Rows:			
	id	nome	preco
▶	1	Notebook	3600.00
	2	Mouse	150.00
•	NULL	NULL	NULL